

Truth Analytics — Design da Plataforma de Análise Multi-Marketplace

Data: 2026-06-10 **Autor:** Matheus Kirsch (Truth Commerce) **Status:** Aprovado em brainstorm — aguardando aprovação da liderança

1. Visão Geral

SaaS da Truth Commerce para clientes de e-commerce/marketplace. A plataforma unifica os dados de venda de todos os canais do cliente (via Bling), coleta benchmark de mercado do nicho (Mercado Livre público + SerpAPI/DataForSEO) e gera, por IA (Claude ou OpenAI), um relatório periódico completo: métricas, gráficos, resumo executivo, gargalos, sugestões de melhoria, ideias de venda no nicho e recomendações de precificação.

Modelo comercial: venda consultiva. A Truth fecha contrato por fora (Pix/contrato), e um admin ativa o cliente no painel interno definindo o plano, que determina a frequência de geração de relatórios: a cada **7, 15 ou 30 dias**. Sem gateway de pagamento no MVP.

Princípio de custo: análise de IA roda apenas no ciclo do plano (relatório periódico completo — formato escolhido sobre dashboard "sempre fresco" ou chat sob demanda). Custo de IA por cliente é previsível e limitado.

2. Decisões de Arquitetura (com alternativas consideradas)

Decisão	Escolha	Alternativas descartadas
Ingestão de dados do cliente	Híbrido faseado: Bling API v3 (OAuth) no MVP ; APIs diretas de marketplaces depois	APIs diretas já no MVP (lento de lançar); só upload CSV (experiência ruim)
Dados de mercado/nicho	ML API pública + SerpAPI/DataForSEO (Shopee, Amazon BR, Google Shopping)	Só ML público (cobertura insuficiente); só IA sem benchmark (sugestão de preço sem base real)
Formato da análise	Relatório periódico completo (formato A) — geração travada pela frequência do plano	Dashboard sincronizado diariamente (custo maior); chat IA sob demanda (custo imprevisível — candidato a fase 2)
Cobrança	Manual: admin ativa cliente e plano no painel interno	Stripe/gateways BR (desnecessário enquanto a venda é consultiva; plugar depois)
Stack	Next.js (App Router) + Vercel + Neon Postgres + Resend	Backend de jobs separado (Railway/BullMQ); VPS próprio — ambos adiam o lançamento sem ganho no volume atual
Orquestração do pipeline	Vercel Workflow (durable workflows)	Cron + máquina de estados própria (retry/recuperação na mão); Inngest (fornecedor extra)
IA	Claude API , saída JSON estruturada validada com Zod	—

3. Módulos

Projeto único Next.js na Vercel, 6 módulos com fronteiras claras. Regra de ouro: **o pipeline só escreve relatórios; o dashboard só lê**. Nenhuma lógica de análise no frontend.

3.1 Auth & Contas

- Cadastro/login com e-mail+senha via Auth.js.
- Usuário pertence a uma `organization` (o cliente da Truth).
- Conta nasce **inativa** (`organizations.status = pending`); só opera após ativação pelo admin.
- Roles: `admin_truth` (equipe interna) e `client`.

3.2 Painel Admin (interno Truth) — `/admin`

- Ativar/desativar/suspender clientes; definir plano (weekly/biweekly/monthly).
- Ver status de integrações, histórico e falhas de relatórios de todos os clientes.
- Protegido por role `admin_truth`.

3.3 Conexões (Integrações)

- OAuth Bling v3; tokens criptografados (AES-256-GCM, chave em env var), refresh automático.
- Cadastro de `tracked_products` (produtos-chave + keywords) que alimentam a coleta de mercado.
- Camada `providers/` com interface comum — preparada para receber APIs diretas de marketplaces na fase 2 sem reescrever o pipeline.

3.4 Pipeline de Relatório (Vercel Workflow)

Disparado pelo botão "Gerar análise" (após validação). Steps duráveis com retry automático:

1. **Coletar Bling** — pedidos do período, produtos, estoque (paginado, upsert idempotente em `orders`).
Roda em paralelo com o step 2.
2. **Coletar mercado** — para cada `tracked_product`: ML público + SerpAPI → `market_snapshots`.
3. **Calcular métricas** — SQL puro, sem IA: vendas por canal, evolução, ticket médio, top produtos, posição de preço vs. mercado.
4. **Análise IA (Claude)** — recebe métricas calculadas + benchmark resumido (dados mastigados; ~2-4 chamadas por relatório). Devolve JSON estruturado: resumo executivo, gargalos, sugestões de melhoria, ideias de venda no nicho, recomendação de preço por produto. Validação com Zod.
5. **Finalizar** — salva jsonb no `report`, `status = done`, seta `proximo_relatorio_liberado_em = hoje + 7/15/30 dias`, e-mail "relatório pronto" via Resend.

3.5 Dashboard do Cliente

- Último relatório: resumo executivo, gráficos (vendas por canal, evolução, ticket médio, top produtos), benchmark de preços, gargalos e sugestões da IA.
- Histórico de relatórios anteriores (imutáveis).
- Botão "Gerar análise" habilitado apenas quando `proximo_relatorio_liberado_em ≤ hoje` e org ativa e Bling conectado.

3.6 Notificações (Resend)

- Conta ativada · relatório pronto · falha de conexão Bling · falha de pipeline (para admin).

4. Modelo de Dados (Neon Postgres)

8 tabelas. `organizations` é a raiz multi-tenant — **toda query filtra por `org_id` da sessão**.

Tabela	Campos-chave	Observações
<code>organizations</code>	status (pending/active/suspended), plano (weekly/biweekly/monthly), nicho (texto p/ IA), <code>proximo_relatorio_liberado_em</code>	A data implementa a trava do plano
<code>users</code>	<code>org_id</code> , email, senha_hash, role (admin_truth/client)	
<code>connections</code>	provider (bling), access/refresh_token 🔒 criptografados, expira_em, status (ok/erro/expirado), last_sync_at	Extensível p/ outros providers
<code>tracked_products</code>	<code>org_id</code> , nome, sku (vínculo Bling), keywords[], ativo	Base da coleta de mercado
<code>orders</code>	<code>org_id</code> , bling_order_id (unique p/ upsert), canal (ml/shopee/amazon/loja...), data, valor_total, frete, itens jsonb	Idempotente — re-rodar pipeline nunca duplica
<code>market_snapshots</code>	<code>org_id</code> , report_id, fonte (ml_publico/serpapi), keyword, dados jsonb	Foto do mercado por ciclo
<code>reports</code>	<code>org_id</code> , período, status (queued/running/done/failed), metricas jsonb, analise_ia jsonb, workflow_run_id	Imutável e auto-contido
<code>audit_log</code>	<code>org_id</code> , user_id, ação, detalhes jsonb	Rastro de ativações, gerações, conexões

5. Tratamento de Erros

Princípio: dado do cliente é sagrado, dado de mercado é desejável.

- **Bling indisponível/token inválido** → falha dura (sem vendas não há relatório). Refresh automático; persistindo, `connection.status = erro`, e-mail ao cliente e ao admin. **Ciclo do plano não é consumido** (a trava só é setada no sucesso do step 5).
- **ML/SerpAPI falham** → degradação graciosa: relatório sai com aviso "benchmark parcial neste ciclo"; IA instruída a não inferir sobre dados ausentes.
- **Claude retorna JSON inválido** → validação Zod; 1 re-tentativa com o erro no prompt; persistindo, o step falha e entra no retry do workflow.
- **Rate limits** (Bling ~3 req/s) → throttling + backoff exponencial dentro dos steps.
- **Falha definitiva do workflow** → `report.status = failed` + e-mail ao admin com step e erro; visível no painel admin.

6. Segurança

- Tokens de integração criptografados com AES-256-GCM (chave em env var).
- Isolamento multi-tenant por `org_id` em todas as queries, com testes dedicados garantindo que cliente A nunca acessa dados do cliente B.
- `/admin` exige role `admin_truth`; rate limiting no login.
- Senhas com hash (bcrypt/argon2 via Auth.js).

7. Estratégia de Testes

- **Unit (Vitest):** cálculo de métricas (parte determinística crítica) e trava de frequência do plano.
- **Integração:** clientes Bling/ML/SerpAPI mockados com fixtures de respostas reais; validação Zod da saída da IA.
- **E2E (Playwright):** cadastro → admin ativa → cliente conecta Bling → dashboard renderiza relatório fixture.
- Implementação via TDD.

8. Fora de Escopo do MVP (fase 2+)

- Gateway de pagamento / self-service (Stripe).
- APIs diretas de marketplaces (Mercado Livre seller, Shopee, Amazon SP-API).
- Chat com IA sobre os dados ("por que minha margem caiu?") — candidato a diferencial do plano top.
- Dashboard com sincronização diária de métricas.
- Multi-usuário por organização (times, convites, papéis granulares) — o MVP assume exatamente **um usuário por organização** (quem se cadastrou). O modelo de dados já suporta vários, mas não haverá fluxo de convite.

9. Riscos Conhecidos

- **Vercel Workflow é produto recente** — mitigação: steps finos e estado no Postgres; migrar para Inngest é troca de orquestrador, não reescrita.
- **Custo SerpAPI/DataForSEO cresce com nº de produtos rastreados** — mitigação: limite de tracked_products por plano.
- **Termos de uso de coleta de dados de terceiros** — usar APIs pagas (SerpAPI/DataForSEO assumem o risco de coleta) e ML público oficial; sem scraping próprio no MVP.